# ABSTRACT

Modern CI/CD services are showing incidents through separate alerts, build logs, deployment histories, source changes, application records and health signals. In case of a failure, an actual person has to connect the dots under time pressure to identify the root cause from downstream symptoms and thus avoid taking actions based on weak evidence. RecoverIT implements a configurable and read-only investigation workflow which collects evidence from registered sources. After collection, it validates and normalises it and builds incident context and a timeline. This is followed by generation of possible root-cause hypotheses, ranking them based on deterministic evidentiary factors, and returning a report linked with the evidence. A LangGraph workflow controls the bounded collection rounds and keeps checkpoints so that the investigation can be resumed later. A provider-neutral reasoning interface supports hypothesis generation, while typed contracts and deterministic validation helps in keeping the output within defined limits.

The current implementation also generates remediation recommendations for human review when the diagnosis is sufficiently supported. Its policy layer blocks recommendations when the investigation cannot reach any satisfactory conclusion, the confidence is low, evidence references are invalid or the instructions contains executable or sensitive content. It does not execute infrastructure changes or claims that the service has recovered.



Keywords: CI/CD, incident investigation, root-cause analysis, evidence correlation, LangGraph, remediation recommendations.

CHAPTER 1

# INTRODUCTION

## 1.1 BACKGROUND AND MOTIVATION

CI/CD helps reduce the time gap between code modifications and their impacts on the running application services. Although it enables rapid delivery of applications, CI/CD also enlarges the number of artifacts which have to be investigated in case of any problems. This includes output from CI/CD pipelines describing building and testing processes, artifacts produced during the release with metadata of the version released, Git history with latest modifications, and symptoms in logs/health checks. Artifacts are usually accessed via different tools. The troubleshooter needs to pick up right events, correlate the timestamps, and separate a coincidental change from a possible cause.

An incident starts with a symptom, not a known cause. A faulty health test may be an indicator of an exception that occurred during application operation, an unavailability of some dependency, a failed deployment, or an alteration in configuration. An entry in the logs can be an incident but doesn't offer any root cause. Here it is not just a problem of locating the data. It is also a problem of picking up the right evidence, temporal ordering of evidence, deciding on what is a cause from various possibilities and establishing if there is enough evidence.

The RecoverIT system has been built with the purpose of carrying out this kind of investigation. It treats the source systems as read-only inputs, then it represents their observations through typed contracts, forms a bounded incident context and presents ranked hypotheses with traceable evidence. The Language model, which is used as the brain of the system gives output that is advisory. Deterministic rules validate references, scoring and final recommendations. This modus operandi aims at making the output inspectable to an operator, and keeping the output from being transformed into a command by the free-response model.

## 1.2 PROBLEM STATEMENT

The current approach to incident response frequently depends on engineers connecting all the dots themselves and correlating CI/CD and production evidence manually in a very limited available time. An effective assistant needs to collect

pertinent evidence initially, then track provenance, construct a timeline, assess multiple causes of failure, and acknowledge uncertainty when stopping for lack of evidence. The system must also ensure that any suggested next action remains reviewable by a human. RecoverIT addresses this bounded investigation and recommendation problem, it does not currently carry out any autonomous recovery.

## 1.3 OBJECTIVES

| Objective | Implementation evidence |
|---|---|
| Unify incident inputs | Typed incident & evidence contracts; configurable source registry |
| Preserve source provenance | Normalization, source identifiers, timestamps and evidence records |
| Construct context | Timeline, relationship & context services |
| Rank causes | Hypothesis service, deterministic ranking engine and stopping rules |
| Control operational risk | Read-only adapters and remediation policy validation |
| Provide usable output | CLI report, HTTP API & web interface |

Table 1.1 Project objectives & evidence

Firstly, one must set up clear boundaries of the evidence intake and sources. Canonical event seeds include the affected service and its time of occurrence. The registered adapter specifies its capabilities to make sure that the graph requests the relevant data only. Errors related to collection of data will be considered the result of the source but not of the evidence itself or the cause to stop the whole investigation.

Secondly, one must ensure the integrity and quality of the evidence. Data will be normalized and, at the same time, will keep its identifier and timestamp of the source. The process of redaction and the quality validation will happen before the data goes to the next reasoning phase. Duplicate evidence will be consolidated.

A third goal is that of producing a bounded and verifiable diagnosis. There might be the need for additional data on the investigation graph, get such data, update the context and compare the hypotheses. A deterministic ranking function evaluates independence of source, symptom coverage, time consistency, change consistency, specificity, predictions, contradictions and absence of causation evidence. Stopping rules allow completed outcomes or the ones which are unable to reach a satisfactory conclusion.

The fourth objective is to deliver responsible remediation. A remediation plan is a descriptive recommendation, with prerequisites and verification. Policy checks help to

avoid unsupported changes. The current system has no write capability in the remediation executor, meaning it cannot make the changes itself rather just suggests suitable remediations and thus, makes no claim of post-action service recovery.

## 1.4 SCOPE AND CONSTRAINTS

The implemented scope covers a configured live runtime with real read-only adapters, a provider-neutral reasoning layer, a LangGraph investigation workflow, checkpoint persistence, ranked hypotheses, a report representation and optional recommendation-only remediation. This solution will be useful for the CI/CD process and any incidents for which there are sources that have relevant records. It will depend on the amount of data available from these sources.

This particular solution does not provide for any production accuracy, mean time to resolution reduction or self-healing capabilities. While the earlier designs did propose some self-healing goals, this particular solution purposely fails to implement them before write operations. Rollbacks, restarting services, scaling and any validation of actions taken will be done in later versions.

The implementation is constrained by provider availability and credentials, adapter coverage, finite investigation budgets and the reliability of external records. The model may generate an inaccurate candidate explanation; consequently, schema validation, provenance checks, deterministic ranking and abstention are part of the design rather than optional presentation features.

## 1.5 EXPECTED OUTCOMES AND REPORT ORGANISATION

The expected result of Project I is an operational investigation application with a repository that proves the path from the canonical event to the hypothesis linked with evidence and the safe advice. Chapter 2 provides a review of the literature that influenced the design. Chapter 3 describes the architecture and approaches used. The results of implementation and direct observation of verification are stated in Chapter 4. Conclusions are drawn in Chapter 5.

CHAPTER 2

# LITERATURE REVIEW

## 2.1 REVIEW APPROACH

The provided literature survey consists of sixteen papers about autonomous SRE, language-model agents, RCA of microservices, CI/CD failure diagnoses, observability, and self-healing. While maintaining the scope of the literature survey, this chapter will evaluate every paper based on the methodology, evidence needs, limitations, and relevance to RecoverIT. The numbers quoted in this section relate to the sources cited and do not correspond to this project at all. The review was conducted before the definition of the RecoverIT scope became complete; references to automated repair serve as motivation only.

## 2.2 MULTI-AGENT SELF-HEALING

Manohar, Shukla & Abi (2025) discuss an autonomous self-healing model for Site Reliability Engineering that incorporates multi-agent architecture for automation of incident management process in cloud-native ecosystems. The authors implemented the framework using tools such as CrewAI and Large Language Models including DeepSeek-R1, GPT-4o, and Gemini 2.5 Flash. The framework is organized into several layers. A Kubernetes infrastructure layer hosts both the target application and the autonomous agent system.

An application layer contains the monitored application and a monitoring agent integrated with Middleware.io, which continuously observes application performance and system health. When an anomaly is detected, an alert starts the self-healing workflow. The central Autonomous SRE Agent layer contains the specialized agents that collaborate on the incident, while an external dependency layer connects the system with services such as Middleware.io, Jira Service Management, GitHub, and Langfuse. The multi-agent system contains four main specialized roles.Incident Resolution Manager will be responsible for orchestration and will manage the incident life cycle and will receive the alert and prioritize and delegate tasks to other agents.

Jira Manager will be responsible for incident ticket management, which includes creation, modification of the findings, change ticket status and incident life cycle

management. Root Cause Analyzer will perform the root cause analysis and correlation of application logs of Middleware.io, Kubernetes cluster state, and source code on GitHub. Code Fix Generator will take the output of RCA and generate code changes and create new git branch and submit pull request. The entire workflow is implemented in the paper as a sequential incident resolution process. An incident starts when Jira ticket is created.

The Incident Resolution Manager selects high-priority incidents and delegates investigation to the appropriate agents. The Root Cause Analyzer collects information from the available operational sources and produces an RCA report, which is added to the Jira ticket. After RCA is completed, the Code Fix Generator uses the report to implement the required remediation and creates a GitHub pull request. A human engineer then reviews the generated pull request before it can be merged.

The incident ticket is transitioned to completion after approval and merging. The implementation was developed using Python 3.12 and CrewAI. The agents have been designed using prompt engineering where their duties, objectives, and workflow have been mentioned. The Root Cause Analyzer has been tasked with correlating the data from Jira ticket details, Kubernetes State, application logs, and source code to find the piece of code or configuration responsible for the issue. The Code Fix Generator has been tasked with carrying out the fix plan.

Custom tools based on REST APIs were developed for Jira, Kubernetes, and GitHub interactions, including a Jira MCP Tool, Kubernetes analysis tools, and GitHub MCP Tools. The complete application was containerized and deployed on Kubernetes. An important part of the implementation is its emphasis on auditability. Langfuse is integrated to record agent interactions and LLM calls throughout the workflow. It helps monitor the activities and decisions made within the multi-agent system. This helps inspect the process of resolving the incident after it has been executed.

The human-in-the-loop phase gives another layer of control as the generated code is reviewed before merging into the master branch. The evaluation of this framework is done in such an environment that mimics the real-life production SRE environment. A faulty web application is deployed on Kubernetes and engineered to generate operational failures such as internal server errors, database connection timeouts, and pod crashes. In this project, the environment is integrated with Middleware.io, Jira

Service Management, GitHub and Langfuse so that the complete incident lifecycle can be monitored, checked and evaluated. Gemini 2.5 Pro is used as an independent judge to evaluate the quality of the work and completeness of tasks performed by the agents.

The evaluation reports 71% correctness in agent task distribution, 68% precision for identifying actionable root causes, and up to 96% accuracy for simpler RCA tasks. The Code Fix Generator has successfully produced valid pull requests for 73% of the tested cases. The JIRA Manager achieved 97% compliance with the expected ticket-state workflow, while the average end-to-end execution time was 27.48 minutes. Langfuse recorded 100% of the workflow executions, which helps in providing complete telemetry of the agent interactions. The results show us that the Root Cause Analyzer can correlate data from various sources to create incident analyses, but it is less effective when the complexity of the problems increases.

The Code Fix Generator shows that the system is capable of taking action and modifying code after diagnosing it instead of stopping at an RCA analysis. The authors therefore demonstrate an end-to-end approach in which LLM agents can coordinate operational investigation and software remediation while retaining human approval for the final change. The paper also identifies future extensions to the framework. The authors propose a long-term memory graph so that agents can learn from historical incidents and reuse proven resolution strategies, together with a searchable knowledge base of previous problems. They also specifically identify deeper integration with CI/CD pipelines and correlation of incidents with recent code changes as a way to further improve RCA precision.

For RecoverIT, this paper is particularly relevant because it demonstrates the practical combination of multi-agent orchestration, cloud monitoring, Kubernetes analysis, source-code inspection, automated remediation, pull-request generation, human approval, and auditability.

## 2.3 LLM-BASED RCA AGENTS

Roy et al. (2024) investigate the use of Large Language Model agents for automated Root Cause Analysis of cloud incidents. The paper focuses on the ReAct agent framework, in which an LLM alternates between reasoning and tool-use actions to investigate an incident. The central idea is that an RCA agent should not be limited to

the information already contained in an incident report; it should be able to retrieve additional information and interact with diagnostic tools in a manner similar to an on-call engineer. The authors build their RCA agent around the ReAct framework. Each iteration consists of one reasoning step which yields the next action, tool selection from a predefined action space, observation of the tool's output, and reasoning once again.

The process ends either at reaching a root cause of the problem or when the maximum number of iterations is reached. The model relies on LangChain and the authors constrain the ReAct loop to 20 iterations due to limited resources. The first set of experiments evaluates the performance of the agent in a generalized setup with no access to the team's diagnostic services. The available tools include incident-details tool, which allows for querying the incident description and the historical-incidents tool which returns previous similar incidents. The goal of this experiment is to see how good is the agent in performing RCA with only general information sources.

There are two kinds of retrievers used in the framework for historical event retrieval. Dense retriever is designed using the SentenceTransformers all-mpnet-base-v2 model and Max Marginal Relevance, while the sparse retriever uses the BM25 technique. The agent can generate retrieval queries during its reasoning process. One retrieval variant directly returns retrieved historical incidents to the agent, while another uses a two-step process in which the agent generates a query and then performs question answering over the retrieved incidents. The two-step approach helps separate the retrieval query from the target incident and reduces context-length problems.

Authors make use of a massive internal incident dataset that is gathered through an enterprise incident portal during January 2020 and September 2021. This dataset consists of around 107,000 unique incidents split into training, evaluation, and test sets. In the experiments, 100 evaluation incidents and 500 test incidents are chosen randomly. The historical incidents from the training dataset constitute the retrieval corpus while incident title and description serve as the main source of information for target incidents. GPT-4 with 8,000-token context window is selected as the main language model while GPT-3.5-Turbo is selected to summarize incident descriptions, root causes, and discussion comments in order to decrease context window size and costs.

The ReAct agent is compared with several non-agent baselines such as Retrieval Baseline, zero-shot Chain-of-Thought, and Interleaving Retrieval with Chain-of-Thought. These baselines allow for comparing retrieval, explicit reasoning, and agent-based reasoning approaches. Evaluation includes automatic and qualitative measures. Automatic evaluation makes use of BLEU, METEOR, ROUGE-L, and BERTScore metrics in order to compare generated root causes with reference answers. Since these metrics do not assess factual correctness completely, manual annotations of predictions are provided using categories such as precise, imprecise, hallucinated, lack of evidence, reasoning error, and retrieval error.

This qualitative evaluation is important because an RCA system can generate a response that appears semantically similar to the reference while containing unsupported or misleading details. In the generalized evaluation, the retrieval baseline improves on several lexical metrics as more historical incidents are retrieved, while semantic similarity changes only slightly. The ReAct variants retrieves approximately four unique historical incidents per incident on average, with around two retrieval lookups. The qualitative analysis shows that the ReAct approach has lower hallucination rates in comparison to the retrieval-only baseline and can sometimes filter out historical incidents which are lexically similar but are diagnostically different. But, the ReAct agent also produces cases where it is unable to determine a root cause because the available incident information is not sufficient for its determination.

The authors also investigate whether discussion comments attached to historical incidents can improve RCA. These comments contain records such as diagnostic steps taken by on-call engineers. The experiments show mixed changes in lexical metrics across models but the semantic metrics indicate that adding discussion comments does not change the resulting root-cause predictions significantly. The authors discuss that discussion comments often contain administrative information or diagnostic steps which depend on tools that are not available to the agent, thus this limits the usefulness if the agent cannot reproduce the same investigation process. The second major part of the paper is a practical case study with the Azure Fundamental Team.

Instead of using only generalized retrieval tools, the researchers adapt the ReAct agent to interact with team-specific diagnostic services. They work with the team over

several weeks to understand its RCA process, identify the diagnostic services used by engineers, and develop tools that allow the agent to interact with those services. The case-study agent is given access to several specialized tools. The Database Query Tool helps in forming queries, executing those queries on the database system of the team and analyzing the results received from it. The results are transformed into Pandas DataFrames and subsequently an LLM-powered DataFrame Query Engine is used to answer questions related to those results.

This allows the agent to perform both database retrieval and subsequent interpretation of structured results. The KBA Question-Answering Tool is provided to the agent in order to extract information from the knowledge base articles. The knowledge base articles consist of troubleshooting techniques, database queries, systems information, and other operational knowledge utilized by on-call engineers.

## 2.4 LOG-BASED RCA

LogRCA focuses on root cause analysis in distributed services using log data as the main source of diagnostic evidence. Its relevance to RecoverIT comes from addressing one of the most important problems in automated incident investigation: operational logs contain large amounts of information, but only a small portion may be useful for identifying the underlying cause of a failure. The work therefore supports the project's need for an evidence-selection and evidence-analysis stage rather than simply passing all available logs to an AI model. For the proposed system, logs will be one of several evidence sources used by the incident investigation agent. When an alert is received, the agent should be able to retrieve logs related to the affected service and incident time window, identify the relevant error patterns and connect them with other operational information.

LogRCA is therefore particularly relevant to the log-analysis component of the proposed architecture and to the broader objective of reducing the amount of irrelevant information presented to the reasoning layer. The paper's distributed-service focus also aligns with RecoverIT. However, in the case of cloud native or microservice architecture, a single issue can produce logs in multiple services and components. The error which is visible may have happened in a downstream service but the initiating issue could have occurred somewhere else. It is therefore important to look into the relation between the log and the timeline of the incident.

In case of cloud-native or microservice systems, a single failure could result in multiple logs being emitted from different services and components. An obvious symptom of the failure could happen in one of the downstream services, whereas the initial failure has happened somewhere else. It means that we need to look at the correlation and connection between log events and the complete incident timeline rather than identifying the root cause based on a single error message.

For instance, a database connection error found in the logs would be checked for the fact that the database is really unhealthy, for the recent changes in database configuration, for the correlation of the appearance of such failures and the last deployment. All this is in line with our proposed investigation process. Instead of having the LLM infer a root cause from the raw logs, the system would first gather the appropriate log window, extract events of interest, create a timeline and then use these events together with evidence collected from software delivery and runtime environment..

The resulting incident context can then be passed to the root-cause reasoning component. The paper is therefore relevant mainly to RecoverIT's observability and RCA stages. It establishes a foundation for dealing with logs as structured data that have to be analyzed and filtered in order to be able to use it as input for root cause analysis. Log analysis in the context of the whole project is one of the possible tools available to the investigator, along with Git, CI/CD, deployment, metrics and service health.

After evidence is collected and a root cause is identified, the system can generate a controlled remediation plan, execute an allowed recovery action when appropriate, and verify whether the incident has actually been resolved. It means that LogRCA can help in solving the problem of evidence analysis in the proposed system.

## 2.5 MICROSERVICE RCA SURVEY

This survey gives a general overview of RCA in microservices and is useful for RecoverIT because it talks about many kinds of operational evidence that are required for performing investigations. It outlines different approaches to RCA which can be carried out by means of metrics, traces, logs, multimodal data, and recently emerged Large Language Models. This is relevant to RecoverIT because its main idea is to allow an AI agent collect evidence not from a single source but from multiple sources

(alerts or log messages). RCA is described in the survey as a process that includes gathering data, analysis of the collected data and the identification and validation of the root causes.

Steps required to deal with the incident begin from collecting information on operation and CI/CD of the incident and then analyzing it in order to discover patterns and relations between them and develop some hypotheses on the cause of the incident. The part of the survey related to metrics will be important for the monitoring module of the proposed system. The CPU utilization, memory usage, network traffic, the number of requests and latency will be the metrics which can help to determine what service or component of the system was affected. When speaking about the first suggestion, metrics will help to identify whether there was resource exhaustion and service degradation and whether the system was restored from this state.

In a microservices-based system, the requests from users can traverse multiple services; hence, it becomes tricky to pinpoint the root cause of the fault. The trace information can help in tracing the path of an affected request and correlating faults across the dependent services. The discussion of the use of logs by the survey also plays a crucial role in the original project proposal. Log files hold rich information about various application and infrastructure events, but it is not easy to analyze due to high volumes and differences. The incident agent proposed can thus fetch the pertinent window of log files and correlate the critical error events along with other operational data.

A particularly important part of the survey for RecoverIT is its discussion of multi-modal RCA. The survey points out that metrics, traces, and logs give different perspectives on the same event. This combination gives more powerful proof than just using one of them. Our proposed system extends this multi-source approach beyond runtime observability by also considering Git changes, CI/CD pipeline results, deployment history and configuration changes. This allows the agent to connect an operational failure with the events that occurred in the software-delivery lifecycle.

Overall, this survey provides the conceptual foundation for the RCA portion of RecoverIT. It gives an idea of an approach in which the incident agent combines multiple evidence sources, identifies relationships between events and services, finds the likely root causes and uses the resulting diagnosis as the basis for a controlled

recovery action. RecoverIT includes this RCA capability as a part of a larger CI/CD incident response workflow, which continues with remediation and post-action verification.

## 2.6 CAUSAL MICROSERVICE RCA

In this paper, the Root Cause Analysis in relation to microservices is investigated from the angle of causal inference. The importance of this work for RecoverIT can be seen in its solution of one of the most important problems in incident analysis, namely the fact that the component exhibiting some abnormal behavior is not always the component that triggered the incident. Since in microservice systems incidents can spread across services via service dependencies, the establishment of a relationship between two incidents does not necessarily prove their causality. This point is very important for RecoverIT, since the incident agent should analyze the underlying cause of the incident rather than report on the service that generated the largest number of errors. For instance, the introduction of wrong configuration in one service by deployment could lead to database or dependent services' failures.

A causal perspective encourages the investigation to consider the dependency structure and direction of the failure rather than treating every correlated signal as an equally likely root cause. The paper is thus relevant to the root-cause-analysis module of the proposed system and its use of evidence from distributed services. The agent can combine operational observations with information about service relationships, recent changes and incident timing to determine which component or change is more likely to have initiated the failure. This complements RecoverIT's timeline-based investigation, where events such as a Git commit, build, deployment, configuration change and the subsequent service degradation are arranged in order to help differentiate between possible causes and the effects that occur afterwards. The paper also highlights that causal RCA methods have different assumptions and strengths and that no single causal approach can be expected to perform equally well for every microservice failure scenario.

It is crucial to this project proposal for the agent designed to use the proposed RCA algorithms to not depend exclusively on one single RCA algorithm. Causal reasoning can be used in combination with log data, metrics, traces, deployment history, Git diffs, and CI/CD as part of an evidence-based approach. This ties into the

hypothesis-based reasoning planned for the project. The agent will be able to produce different hypotheses concerning the problem in question and use further evidence to confirm which one of these hypotheses fits the order of events better. Causal information will make this process easier by allowing the agent to identify the true cause of the event among subsequent symptoms caused by dependencies.

It is also related to the RecoverIT evaluation technique, as it proves that RCA methods must be checked under various types of failures instead of taking for granted that an efficient RCA approach for one type of incident will be effective for all others. Therefore, for our controlled prototype, it justifies the usage of various incident scenarios, namely deployment regression incidents, configuration incidents, service incidents, and dependency incidents. In relation to RecoverIT project, its primary significance lies in the causality aspect of RCA.

With the help of the described combination of this perspective with multi-source evidence gathering and CI/CD context, it becomes possible to use the gained diagnosis for selecting a controlled remediation action and verifying if the incident was handled.

## 2.7 RCA BENCHMARKING

Pham et al. (2025) present RCAEval, an open-source benchmark aimed at solving one of the key problems associated with RCA for microservices: the absence of large-scale and reproducible data as well as the absence of an evaluation framework. Specifically, the authors note that there is a tendency among researchers to evaluate their techniques in terms of performance on a few systems and few kinds of faults, which makes a direct comparison of different approaches very difficult and does not allow researchers to gauge the generalization ability of their approaches. RCAEval focuses on shifting the attention from developing another RCA algorithm to creating a common ground upon which different RCA algorithms can be created, compared, and reproduced. RCAEval includes 735 cases of failures in three microservices: Google Online Boutique, Weaveworks Sock Shop, and Train Ticket. What is important, the benchmark includes multiple telemetry modalities: metrics, logs, and traces.

These datasets include RE1, RE2, and RE3. RE1 consists of metric-based failures, whereas RE2 is an extension of RE1 by taking into account multi-source RCA based on metrics, logs, and traces. RE3 considers code-based failures, while providing

multi-source telemetry. Every failure instance includes information about the root-cause service as well as the indicator for the root cause such as a certain metric or log that helps in conducting both coarse and fine grain localization. Thus, the benchmark can provide a much more comprehensive basis for RCA evaluation than those which rely on a single telemetry source or synthetic data.

The paper is particularly useful for RecoverIT as the project approach is also focused on the multi-source evidence correlation approach for incidents analysis. The following evidence sources are listed in the project specification: application and infrastructure logs, logs from CI/CD pipelines, monitoring metrics, deployment history, Git commits, pull requests, configuration changes and information about the health of services. RCAEval demonstrates the significance of correlating heterogeneous operational evidence rather than relying on one signal. The use of metrics, logs, and traces makes it possible to build an excellent base for the telemetry analysis layer of the project. In addition to that, the project focuses on Git history, deployment and CI/CD history and broadens the context of incidents to the software delivery life cycle.

In distributed microservice systems, a fault in one component can be passed through service dependencies and create symptoms in various downstream services. Because of this, the most visible and most obvious anomaly is not always the actual fault which started the issue. This is closely related to the investigation workflow of the proposed agent, where the system is expected to construct a timeline, generate multiple root-cause hypotheses and evaluate supporting and contradicting evidence instead of immediately selecting the first observed error. RCAEval's annotated root-cause indicators provide a suitable reference which helps in evaluating whether an autonomous investigator can identify the underlying cause rather than just detecting abnormal services. Another important contribution of RCAEval is that is has a reproducible evaluation framework.

The paper provides 15 baselines covering metric-based, trace-based and multi-source RCA and it also supports evaluation at both coarse-grained and fine-grained levels. This is important for RecoverIT because an autonomous incident agent needs to be evaluated systematically rather than only through qualitative demonstrations. RCAEval provides an approach for controlled experiments where known faults are

injected, telemetry is collected and the predicted root cause is compared with the ground truth. The proposed system could adopt this approach of evaluation while extending the measured outcome from RCA alone to the complete incident response workflow. RE2 contains very large amounts of telemetry, including millions of log lines and millions of traces for the benchmark systems.

This supports an important design principle for the proposed agent, which is that it should not blindly pass all the available operational data to an LLM. Instead, it should selectively retrieve and summarize the evidence relevant to the incident before passing it to the LLM. This is directly related to the project's Incident Context Builder and Data Collector, where the agent is expected to identify what information is missing, query relevant data sources and construct a structured incident timeline. The same approach of reducing the evidence is also seen in other supplied literature, where LogRCA tries to identify a minimal set of causally relevant log lines, while MicroRCA-Agent applies filtering and summarization before multimodal reasoning. RCAEval's inclusion of code-level faults is also relevant to a CI/CD oriented incident trigger.

The RE3 benchmark is based on faults inserted into the application's source code, and utilizes telemetry like logs and traces for their diagnosis. This illustrates that symptoms observed in production systems may be traced back to the problems in code rather than problems in the infrastructure alone. For RecoverIT, this implies that there is a need to correlate telemetry with recent Git check-ins, pull requests, deployments, and configuration changes. As such, a CI/CD incident can be understood as a series of events starting from the change in the source code to deployment and finally to symptoms, and not just as an isolated incident detected through monitoring.

Wang and Qi's survey classifies microservice RCA into metric-based, trace-based, log-based, multi-modal and LLM-assisted approaches, highlighting the need to analyze different types of operational evidence. Roy et al. show that LLM agents can improve RCA by dynamically retrieving additional diagnostic information rather than relying only on an incident description. These findings are directly applicable to the proposed system's design: evidence collection should be tool-driven, reasoning should be explicit, and autonomous actions should be constrained by safety policies.

## 2.8 AUTONOMOUS CLOUD EVALUATION

The work by Chen et al. (2025) introduces AIOpsLab as an integrated design, development, and evaluation framework for AI agents that support autonomous cloud operations. The motivation behind the work is attributed to the rise in complexity of cloud and microservices environments, together with the lack of capability of existing AIOps benchmarks in assessing AI agents on the entire lifecycle of incidents. Instead of assessing an AI agent either on static datasets or operational task, AIOpsLab provides an interactive environment where the agent could interact with the running cloud system by observing, investigating faults, taking actions, receiving feedback, and getting evaluated against a predetermined expected result. The wider paradigm introduced in this paper is referred to as AgentOps, which allows AI agents to conduct various cross-layer operations rather than detection/classification task.

The system includes a microservice cloud infrastructure, a workload generator, fault injection capability, telemetry capture mechanism, an orchestrator, and the Agent-Cloud Interface (ACI). ACI acts as a control interaction interface through which the agent can interact with the cloud environment, perform actions, and receive feedback from the cloud. Thus, the incident response scenario in this framework involves a sequence of interactions rather than a single step prediction, which has high relevance to RecoverIT as RecoverIT also involves the same loop for investigation: incident arrives, evidences collected, hypotheses formulated and tested, recovery plan/action made and system state verified. In AIOpsLab, the operational problem is modeled as P = <T, C, S> where T stands for task, C for context and S for oracle.

The task taxonomy comprises four levels: detection, localization, RCA, and mitigation. Detection is the determination of the existence of a fault; localization determines which service or component has been affected; RCA detects the layer of the system at which the fault occurred, and the type of the fault; while mitigation involves interaction of the agent with the environment to fix the fault, such as fixing the configuration or reverting to an earlier version. In addition, it reinforces the theory that incident management should be judged as a series of increasingly difficult operational tasks rather than on the basis of RCA scores. The evaluation framework uses two microservices applications from DeathStarBench as case studies.

The tools that have been used for metrics, distributed tracing, and logging are Prometheus, Jaeger, and Filebeat/Logstash respectively. The Kubernetes and Helm API is part of the orchestrator. This multi-source telemetry design is directly relevant to RecoverIT because our incident investigator is expected to reason over logs, metrics, service health, deployment information, and other operational evidence. AIOpsLab demonstrates how an agent can interact with telemetry in a live environment instead of receiving a fixed, pre-collected incident record. Another important contribution is the benchmark itself.

The paper proposes 48 evaluation problems that focus on various AIOps use cases and evaluates multiple LLM-based agents. Unlike previous benchmarks, which focused on incidents involving only performance degradation or application crash, the proposed one uses more sophisticated scenarios and precise root causes. The evaluation framework takes into account not only accuracy but also efficiency metrics like Time-to-Detect (TTD), Time-to-Mitigate (TTM), number of agent actions/steps, and token consumption. This particular evaluation method is relevant to RecoverIT as our intended evaluation not only should show that the agent has identified the root cause accurately but also should take into account the time and efficiency of investigation, success of the remediation process, and number of actions performed.

The paper reports that the RCA and mitigation tasks are the most challenging for the evaluated agents, and that none of the agents consistently achieves high performance across all four task categories. The authors also show that increasing the number of agent steps can improve performance for some agents, but that performance may eventually plateau. In the reported experiments, FLASH reaches its highest accuracy of 59.32% with a step limit of 20, while increasing the step limit for GPT-3.5-Turbo beyond a certain point mainly increases token consumption without improving accuracy. These findings support RecoverIT's emphasis on purposeful investigation rather than unrestricted agent exploration. AIOpsLab also identifies failure modes that are important for the proposed system.

All these notions are consistent with the safety layer implemented by RecoverIT, where the AI system should not be given unfettered access to infrastructure and remediation should go through policy, allowlist, approvals, dry run, and post-execution check. Moreover, this paper has significance from an evaluation

standpoint since it asserts that static telemetry data sets and static Q&A benchmarks do not capture dynamic cloud activity well.

A real incident changes the state of the environment as investigation actions are performed, and a mitigation action can itself change the subsequent evidence available to the agent. AIOpsLab addresses this by allowing agents to interact with an environment and by evaluating the final system state.

## 2.9 SOP-GUIDED RCA

Pei et al. (2025) propose Flow-of-Action, a Standard Operating Procedure (SOP) enhanced LLM-based multi-agent system for root cause analysis in microservice environments. It should be noted that the paper highlights a significant drawback of general-purpose LLM agents that can be applied to RCA, despite the fact that the ReAct paradigm is similar to how Site Reliability Engineers analyze issues; an LLM may hallucinate, choose inappropriate tools, and even be overloaded by various and complicated observations.These errors can affect the subsequent investigation steps and lead the agent away from the actual root cause. Flow-of-Action thus focuses on controlling and structuring the investigation process rather than allowing the LLM to freely select every action. The main idea is to encode the diagnostic procedures followed by experienced SREs as Standard Operating Procedures and use them as guidance during RCA.

The authors introduce an SOP centric framework called SOP flow. It contains tools that identify the SOPs relevant to an incident or automatically generate an SOP when a suitable procedure is not available, and convert SOPs into executable code. The SOP acts like a soft constraint and guides the main agent towards the correct path for diagnosis, it is not completely rigid and thus also allows some flexibility in case something unexpected occurs. This is important for RecoverIT because the project also requires an agent to investigate incidents dynamically, but within a controlled and explainable workflow rather than relying on unrestricted LLM reasoning.

The paper further mentions to divide responsibilities among multiple agents. The MainAgent is responsible for coordinating the RCA process, while CodeAgent assists with code-related investigation, JudgeAgent is responsible to determine whether the root cause has been identified, ObAgent extracts important information and fault

types from large amounts of observational data, and ActionAgent assists in generating and selecting action sets. This separation of responsibilities is directly relevant to RecoverIT's agentic architecture. Our system can similarly benefit from specialized tools and modules for log retrieval, Git analysis, CI/CD analysis, deployment history, root-cause reasoning, recovery planning and verification. Flow-of-Action demonstrates that distributing complex RCA responsibilities can reduce the cognitive burden on a single LLM and this can also improve the overall investigation process.

The experimental results demonstrate the value of these constraints. The paper reports that the ReAct baseline achieves 35.50% RCA accuracy, while Flow-of-Action achieves 64.01%. The authors attribute this improvement to the combination of SOP-guided orchestration, the action-set mechanism and multi-agent collaboration. Ablation results further show that removing individual agents reduces accuracy to some extent, while removing the action-set mechanism has a more noticeable effect on action efficiency. The paper also sets a limit on the number of investigation steps that can be carried out to prevent unbounded loops.

This paper is highly relevant to the suggested framework since it confirms that the RCA agents must be built to ensure controlled reasoning, bounded tool usage and well-defined termination criteria. Another framework, Flow-of-Action, is relevant to the other pieces of literature as well. RCAEval is a benchmark for evaluating RCA through metrics, logs and traces in several microservices, whereas AIOpsLab is a platform for testing the agents in an interactive cloud computing environment where they are able to detect, localize, diagnose and repair faults. LogRCA tackles the challenge of collecting useful logs out of a large number of logs, and the LLM-agent framework highlights the importance of allowing the agent to collect diagnostic information. Finally, the Flow-of-Action framework adds one more component to the list: the structured operational knowledge needs to steer the process of using the tools by the specialized agents.

Together, these works help in RecoverIT's architecture of evidence collection, structured reasoning, controlled action selection and verification. For the Autonomous CI/CD Incident Triager & Self-Healer, Flow-of-Action is especially relevant to the investigation and decision-making layers.

## 2.10 INTELLIGENT FAULT DETECTION

Ji and Luo (2025) propose a cloud-based AI framework that combines Large Language Models (LLMs) with machine-learning-based fault detection to support intelligent fault detection and autonomous self-healing in cloud systems. The motivation behind this paper comes from the growing size and complexity of cloud computing infrastructure where a rule-based detection mechanism as well as human intervention becomes increasingly hard to achieve. The suggested framework uses the semantic capability of an LLM in processing system logs and error reporting as well as real-time data streams, whereas traditional machine learning approaches have been used for fault classification and anomaly detection. The framework follows a multi-level architecture.

The method uses supervised learning for fault classification along with unsupervised learning for detecting anomalies, thus enabling the framework to detect abnormal behavior and classify fault conditions. The LLM module helps understand semantics of the operational data like logs and error messages, while the machine learning modules enable structured detection capability. Once the fault is detected, a self-healing process is automatically initiated by the framework. This technique is applicable to RecoverIT since our framework also requires a combination of analysis of the operational data along with higher reasoning apart from just using the rule-based approach.

There are some cloud events that create text artifacts like logs, errors, warnings, and reports that do not suit anomaly detection through numbers in an effective manner. Here, the natural language processing ability of the LLM becomes handy while analyzing the information in a semantic sense. In RecoverIT, it helps use the LLM as the reasoner for the analysis of the different types of evidence. This is made possible by adding extra information apart from the application logs and CI/CD logs like git commits, deployment logs, configurations, and alert messages describing the incident in a different way.

Self-healing systems are meant to automatically react to faults through corrective measures such as service restart, resource allocation, or corrective changes. According to Ji and Luo, detection alone is insufficient since for a practical reliability system in the cloud environment, downtime needs to be reduced by recovery in addition to just

detection. The approach adopted by Ji and Luo connects fault detection to the process of automatic self-healing. In relation to RecoverIT, this refers to the use of the Recovery Planner and Safe Action components, which involve converting the suspected cause into a planned recovery operation like a rollback, restart, scaling or retry.

The paper's use of multiple operational data sources is also closely related to our proposed system's observability layer. The original project proposal is designed to collect application and infrastructure logs, CI/CD pipeline logs, monitoring metrics, deployment history, Git changes, configuration changes and service-health information. Ji and Luo specifically emphasize logs, error reports and real-time data streams as inputs to the LLM based framework. This supports the project's idea to consider operational information as a collection of different types of evidence rather than relying on a single monitoring signal.

The paper is complementary to the other supplied RCA literature. RCAEval shows the significance of RCA evaluation from multiple telemetry streams and practical microservices fault cases. On the other hand, LogRCA is dedicated to finding the limited root-cause information among the huge quantity of log data. Roy et al. demonstrate that LLM agents can dynamically retrieve additional diagnostic information during RCA. AIOpsLab evaluates agents in interactive cloud environments and shows that complete RCA and mitigation are more difficult than simple detection or localization.

Flow-of-Action also shows how investigation using LLM requires the use of procedures and limited action selection. These two papers indicate that there is need for the suggested system to consider both detection, evidence gathering, reasoning, action selection and verification rather than taking self-healing to be an automatic reaction to a warning signal. Ji and Luo also note that there is improvement in fault detection compared to conventional techniques. These outcomes are directly related to the evaluation goals of RecoverIT. Our system should not be evaluated only on whether it can generate a plausible root-cause explanation.

## 2.11 AGENT-BASED MICROSERVICE RCA

Overall, MicroRCA-Agent demonstrates that effective LLM-based RCA in microservice environments depends not only on the reasoning capability of the LLM, but also on the quality, compression, alignment, and integration of the evidence provided to it. Its modular multimodal architecture provides a strong reference for RecoverIT's evidence-driven RCA layer. RecoverIT extends this approach by also considering CI/CD and source-change context, agentic evidence retrieval, hypothesis-based reasoning, controlled remediation and closed-loop verification. This allows the system to connect multimodal RCA with the complete incident-response and self-healing workflow. The paper also supports the project's evaluation strategy. MicroRCA-Agent demonstrates the value of ablation experiments for understanding which evidence sources contribute to RCA.

A similar approach can be used in RecoverIT by comparing an agent with logs only, logs plus metrics, logs plus metrics plus traces, and finally the complete CI/CD-aware evidence set including Git and deployment information. In addition to root cause accuracy, the next version of RecoverIT may analyze investigation time, evidence quality, number of investigation steps, remediation accuracy, false remediation and recovery time. In terms of RecoverIT, MicroRCA-Agent will be especially useful for the following modules: Evidence Collection, Incident Context Builder, RCA Engine. The architecture of the project could borrow the modality-specific preprocessing idea from it, i.e. logs could be preprocessed by parsing and filtering; metrics - by anomaly reduction; traces or service health information could be summarized before submitting to the reasoning agent. Then the project broadens the operational context with CI/CD specific evidence, such as recent git commits, pull requests, deployment history, pipeline execution results and configuration changes. Thus, the agent is able not only to understand where the failure happens, but also what has been changed right before that.

The paper's results also highlight the importance of managing LLM context and hallucination risks. MicroRCA-Agent uses filtering, summarization, structured prompts and modular processing to control the information provided to the LLM. The reported analysis includes cases where LLM reasoning can still produce hallucinations, showing that multimodal LLM-based RCA is not automatically

reliable just because multiple data sources are provided. This is relevant to the safety architecture of the proposed system. The project should constrain the LLM through predefined tools, structured evidence, action allowlists, policy checks and post-action verification rather than allowing it to perform unrestricted infrastructure actions.

One more thing that should be said about MicroRCA-Agent is that this tool can provide a very good understanding of the use of LLMs in operational applications. Specifically, using a large language model does not imply replacing any other techniques of operation processes monitoring and anomaly detection. In such a way, standard techniques are applied to collect and categorize operational data, whereas LLMs become relevant in case the semantics analysis and multimodal analytics are required.

Logs can provide descriptions of failures, metrics can identify quantitative deviations, and traces can show distributed request behaviour. Combining these signals provides a more complete incident context. The Multimodal RCA module acts as the main decision-making stage. It combines the processed log features, anomalous trace information and summarized metric phenomena, and uses carefully designed cross-modal prompts to reason over the combined evidence. The output includes the faulty component, root-cause description and reasoning trace. It also uses a two-stage LLM analysis to summarize phenomena hierarchically across service, pod, and node levels.

This is especially relevant to RecoverIT's Incident Context Builder. Our solution needs to do exactly the same thing, turn monitoring data into the explanation of what has changed, where the changes happened, and how the incident was affected by this. Also, the hierarchical analysis suits well the architecture of cloud native applications, which generate incidents and symptoms on multiple levels at once.

## 2.12 MULTI-AGENT RCA SEARCH

Naakka, Wang, and Mäntylä (2026) propose LATS-RCA, an LLM-based multi-agent framework for root cause analysis in microservice systems. This paper deals with the challenges of RCA in the manual setting of distributed microservices, wherein engineers have to correlate signals of operations and analyze multiple responsible services within a tight deadline. A weakness in previous works on RCA using LLM is

found by the authors: all such approaches employ a linear reasoning strategy, which is inappropriate when the first appearance of an anomaly suggests that there are multiple services/components behind it. This challenge is solved in LATS-RCA through formulating the task of RCA as a tree search guided by reflection. The core of LATS-RCA is the Language Agent Tree Search mechanism, combining LLM reflection with Monte Carlo Tree Search.

Multiple LLM-driven agents iteratively reason over execution logs and performance metrics, while the search maintains competing hypotheses and expands promising diagnostic paths. Reflection evaluates intermediate reasoning states using evidence quality, diagnostic completeness, and internal consistency. This is highly relevant to RecoverIT because the original project proposal also intends to generate several root-cause hypotheses and rank them using supporting and contradicting evidence rather than selecting the first plausible explanation. The architecture contains specialized log and metric agents coordinated by a supervisor. Each agent investigates its own observability modality, while the supervisor controls sequencing, evaluates progress, performs cross-modal correlation, and can hand off a condensed evidence summary when additional analysis is required.

It is highly related to the architecture of RecoverIT in which the various capabilities deal with logs, metrics, Git, CI/CD, deployment history, RCA, recovery planning, and verification. In this paper, the importance of specialization with respect to central coordination is shown. One important aspect in particular is the presence of multiple candidate hypotheses. Instead of focusing on one chain of thought for RCA, LATS-RCA takes into account different alternatives and decides which should be investigated by reflection. It fits in with the RCA Engine proposal in which the increase of HTTP 500 errors might be due to a faulty deployment, database issue, configuration problem, and so on.

The agent can collect additional evidence and update hypothesis confidence before recommending remediation. On the LO2 benchmark, LATS-RCA achieves 91.3% diagnostic accuracy, compared with 39.8% for single-agent ReAct and 57.4% for multi-agent ReAct. The ablation study shows that candidate batching, backpropagation, and reflection all contribute to performance, with removal of candidate batching reducing accuracy to 84.3%. These results support the use of

structured hypothesis exploration rather than simple linear reasoning. But, the improvement in accuracy also causes higher costs of computation.

LATS-RCA averages 53.1 API calls, 156K tokens and 9.1 minutes per investigation, compared to 13.5 calls, 41K tokens and 1.2 minutes for single-agent ReAct. It explores an average of 18.9 hypotheses per investigation. This is important for the original project proposal because an incident response agent cannot just focus on maximizing reasoning depth. The investigation also needs to have bounded tool call, token and time budgets, along with a proper criteria to stop.

The evaluation of the LATS-RCA tool took place in a heterogeneous production environment, in which the microservices were Node.js, Rust, Python, and Go. The LATS-RCA tool had accuracy rates of 60-70% for 37 production instances, which were less accurate than the 91.3% accuracy rates achieved through LO2. The above production environment was heterogeneous concerning the logs, timestamps, severity levels, metric names, and coverage. This undoubtedly implies that heterogeneity is an essential factor in the agentic root cause analysis process. Based on this, one of the core design principles of RecoverIT is to correlate the evidence before any reasoning can be made.

The paper also provides useful guidance for the project's evaluation strategy. In addition to root-cause accuracy, the project should measure investigation time, tool/API calls, tokens, number of hypotheses explored, remediation success, false remediation, and recovery time. This is important because LATS-RCA demonstrates that higher diagnostic accuracy can require substantially greater computational cost. In summary, Multi-Agent Systems for Root Cause Analysis in Microservices shows how using structured multi-agent searches for competing hypotheses can greatly benefit RCA relative to linear analysis, along with its associated costs and heterogeneity problems. It is an excellent basis for the hypotheses generation and reasoning layer of RecoverIT, although the initial project proposal ties the reasoning to CI/CD-based investigation, constrained remediation, and verification in the loop.

## 2.13 GRAPH-AUGMENTED RCA

Tian et al. (2026) present GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices. The article deals with solving three

interrelated problems of RCA of microservices: firstly, the root cause can be different from the service where the problem symptom is located; secondly, different telemetry modalities may reveal different potential root causes; thirdly, mere discovery of the failing service will be insufficient for the SRE since he will require both the explanation and a course of action. Thus, GALA combines service dependency graphs, various kinds of telemetry modalities, statistical hypothesis formation, and agentic investigation by the means of the LLM. In such a way, GALA is highly relevant to the task of RecoverIT since it implies correlation of various types of operational data and transition from the stage of detection through diagnosis to the stage of controlled recovery. In GALA, metrics, logs, and distributed traces are used as complementing evidences.

Metrics are numeric measurements (CPU utilization, latency); logs are timestamped events; traces are inter-service calls and their temporal relations. The article uses distributed traces to construct a service dependency graph for the agent to reason about failure propagation topology. It is very important for the task of RecoverIT since an incident may occur in one component and its root cause may be located in it.In our framework, we can apply this concept not only in regard to runtime service dependencies but also extend it to relationships between CI/CD pipelines, deployment, Git changes, config changes, and affected services. The four phase architecture in GALA+ consists of:

Phase I: Root cause hypothesis generation using complementary metric and trace based signals. Phase II: Generation of pod level diagnostic packages containing relevant telemetry data in their temporal and dependency context. Phase III: Graph driven agentic investigation. Phase IV: Generation of diagnosis, incident report and action items.

This staged procedure aligns well with the workflow implemented in RecoverIT for incident detection, data gathering, AI analysis of incidents, RCA, recovery plan creation, execution of safe actions and post-incident reporting. One of the major contributions is the implementation of hypothesis generation using dual signals.

The BARO framework in GALA+ provides coarse ranking using metrics while STRIX provides graph-structure aware ranking. Both rankings are combined into one consolidated list of candidates prior to LLM agents' deeper investigation. What

explains the reason why the log file was not taken into account during the first stage of the ranking process is that sometimes failure may be silent, leading thus to a bad quality of logs; logs can be taken into account only at a later stage of the diagnostic process. This way of approaching the problem is a very important part of the project's initial design where no evidence should be over-represented.

Raw telemetry is distilled into a compact representation that preserves temporal patterns, dependency context, and error signals. This reduces the amount of information that must be processed by the LLM and allows the agent to focus on evidence relevant to each candidate service or pod.

An Investigator Agent assesses a candidate based on the diagnostic package and comes up with an assessment consisting of confidence, evidence, cause, and supporting and contradicting factors. In case of insufficient confidence, the agent investigates further via the dependent services in batches. This way, the search process becomes constrained: the agent will not check all services at random; it will go along the most probable fault-propagation routes and stop once it gathers enough evidence. The method corresponds to the RecoverIT need for generation of several hypotheses regarding the root cause and their ranking based on evidence. For example, in the case when some problems happen after a deployment process, the problem can be a result of the following factors: poor code change, configuration change, database problems, resources depletion, or dependency problems.

Instead of choosing the best option immediately when there is an anomaly in the system, our agent is able to maintain multiple hypotheses, collect facts for all of them, and choose the best one depending on the dependencies and the timing. GALA+ also demonstrates why graph-guided exploration can be preferable to unrestricted LLM tool use. It is indicated in the paper that the previously proposed GALA approach had the possibility to acquire inadequate data due to the fact that it was studying only one telemetry packet at a time, whereas unbounded ReAct search could go into an irrelevant direction, deviating from the possible root causes. The graph therefore functions as a reasoning constraint rather than merely as a retrieval structure.

The results show strong performance. In the Online Boutique and Train Ticket datasets, GALA+ performs at an AC@1 of 74.44% and 73.33% and beats the strongest LLM-based baselines by more than 25 percentage points. Another dataset

that is examined in the paper is AegisLab, which features 25 failure modes, and in which GALA+ reports the highest MRR, AC@1 and AC@3 scores among all tested approaches. This demonstrates the merit of leveraging graph-based structure, multimodal cues, and agential investigation as opposed to a single telemetry stream or unrestricted LLM reasoning. The paper does not evaluate only root-cause ranking.

It also evaluates incident summarization and action recommendations using SURE-Score, a human-guided evaluation framework developed with SRE expertise. SURE-Score assesses dimensions related to the usefulness and quality of incident reports, and human SRE evaluation confirms GALA+'s advantage over Flow-of-Action across the evaluated dimensions. This is particularly relevant to the original project proposal because the intended output is not just a root-cause label.

## 2.14 EXCEPTION-BASED RCA

Mhatre and Shaikh (2025) propose an automated Root Cause Analysis (RCA) system for microservices architectures using Large Language Models combined with a machine-learning-based exception classifier. The paper concentrates on the huge number of business and runtime exceptions in microservice-based applications and the inefficiencies of exception analysis manually. The main contribution of the paper is in developing a hierarchical approach to aggregate, categorize the exceptions and use tuned LLMs to conduct code-aware RCA using runtime stack trace and source code context.This technique is especially applicable for RecoverIT since it helps illustrate the connection between runtime exceptions and application code using LLMs instead of considering any event just another deviation in monitoring data. The process starts from the creation of an exception aggregation layer based on Apache Kafka.

Exception messages raised in various microservices are placed in a Kafka topic and are then passed to an Exception Classifier. This classifier preprocesses error messages and categorizes exceptions into business exceptions and runtime exceptions. Runtime exceptions are then sent to the RCA service. This is helpful because not all exceptions need to go through the same diagnostics process, and by filtering the stream of events prior to passing the information to the LLM, we can avoid unnecessary processing. In the case of RecoverIT, the same technique of staged processing can help separate

informational events, application errors, deployment problems, and incidents needing investigation.

The Exception Classifier compares traditional machine-learning approaches including XGBoost, Support Vector Machine, and Naïve Bayes with LLM-based classification. The paper reports that XGBoost performs best among the selected conventional classification models. The experience from this research may be useful for further work on the proposed system since an LLM doesn't necessarily have to substitute all conventional methods. Deterministic algorithms and machine-learning techniques will take care of the classification tasks and LLM will be utilized only for the tasks that require reasoning and context analysis. Such design emerges from the design of the entire system when operational tools collect information and then AI engine correlates and analyzes the information.

The RCA solution is built using CodeLlama and Mistral models that have been fine-tuned. It receives runtime exception stack traces together with tokenized or preprocessed code-repository information and generates a root-cause analysis. The paper reports that CodeLlama-7B performs particularly well when supplied with preprocessed code snippets. The approach therefore establishes an important connection between runtime symptoms and source-code context. For the proposed CI/CD incident triager, this is directly relevant because recent Git commits and pull requests can be used as evidence when an incident occurs after a deployment.

Instead of asking only “which service failed?”, the agent can investigate “what changed in the code or configuration immediately before the failure?” The paper's layered architecture also has a natural correspondence with RecoverIT's workflow. Kafka-based exception aggregation corresponds to the project's Incident Detection and Data Collection layer; exception classification corresponds to initial incident triage; the LLM-based RCA service corresponds to the RCA Engine; and the repository-aware analysis provides a foundation for correlating runtime symptoms with software changes. RecoverIT extends this workflow beyond exception-driven diagnosis by incorporating monitoring metrics, deployment history, CI/CD pipeline results, configuration changes, and service-health information. The biggest merit of the work is in its code-aware approach to diagnostics. While stack traces will show where the exception took place, in order to find out what really happened, it is

necessary to analyze the corresponding piece of code and interactions between the functions/components.

Thus, through the use of preprocessed pieces of code, the system tries to shift from exception classification to explaining the cause. It is especially important for RecoverIT since production issues can be caused by a wrong code change despite being manifested as generic HTTP errors, timeouts, and/or services not working even though the symptoms are clearly observable. The paper states that more than 90% precision in classification of business exceptions, 89.6% recall in runtime exception classification, less than one second per exception for RCA diagnostics, and more than 85% accuracy in human qualitative analysis have been achieved. In this way, the potential of using automatic classification and code-aware LLM diagnostics can be seen. For RecoverIT, speed is especially important.

However, our assessment has to be extended to include the accuracy of multi-source RCA, the investigation process, the success of the remediation, recovery time, and verification reliability. There are a number of research directions that are related directly to the suggested solution. In particular, the authors understand that their existing RCA tool is based on exception stack traces and code context and consider the possibility of using environment logs including resource consumption and network/disk activity. Moreover, they suggest feedback mechanisms for adjusting the model according to changes in the services and mention autonomous remediation as the future direction for the project. All these findings coincide completely with the direction of the initial project proposal, as the suggested agent is supposed to use application and infrastructure logs, metrics, Git commits, deployment logs, CI/CD data, and configuration changes to diagnose problems and move to the next stages.

The paper is complementary to the other supplied literature. RCAEval demonstrates the value of multi-source telemetry and reproducible RCA benchmarks. MicroRCA-Agent uses logs, traces, and metrics through modality-specific preprocessing and then followed by multimodal LLM reasoning. GALA uses service dependency graphs and graph-guided investigation to reason about fault propagation. Flow-of-Action introduces SOP-guided multi-agent coordination to control LLM tool use.

Interactive cloud settings are used for evaluating agents for detection, localization, RCA, and mitigation through AIOpsLab. These papers demonstrate a development from specific fault analysis to self-reliant evidence-based investigations of incidents. The contribution of the paper written by Mhatre and Shaikh is significant in providing a software point of view.

## 2.15 HEALTH-PROBE REMEDIATION

Kalakoti (2025) introduces an AI-enabled self-healing infrastructure architecture that is comprised of continuous health monitoring, incident diagnosis through AI, and predetermined remediation playbooks. This research is concerned with a particular challenge in today's cloud and distributed infrastructure: health monitoring can establish that there is an issue with a particular service, but this is not enough to ensure that the problem will be fixed quickly. Therefore, the solution suggested is a loop that includes health monitoring, incident diagnosis using AI, execution of the predetermined playbooks, and re-evaluation of the situation. This is very much applicable in the context of RecoverIT since it too intends to go beyond simply generating alerts and work towards autonomous recovery. Architecture in this case includes the use of Kubernetes as the underlying infrastructure as well as the inclusion of health probes, monitoring data, reasoning using artificial intelligence, and remediation actions.

The liveness, readiness, and startup probes of Kubernetes offer continuous feedback regarding application health and availability. Where an unhealthy situation is detected, AI-driven reasoning as well as an effective remediation playbook could be applied. It is critical to consider health probes as the triggering approach in RecoverIT since the system requires a means of validating whether the incident is ongoing and whether the remediation operation has indeed brought back the healthy state of service. One major contribution is the integration of AI reasoning together with the predetermined remediation playbooks.

The structure of the paper can be defined as a loop during which health monitoring takes place, anomaly detection happens, the reason for the problem is discovered with the help of AI analysis, a plan of actions is created and carried out, and finally, health is checked again to see if everything is done properly. It is pretty much similar to

RecoverIT, including incident identification, evidence gathering, root cause analysis, recovery planning, action execution, and results verification.

In case the health does not improve, the agent will consider the remediation as failed and continue investigating instead of ending the incident automatically. The framework also applies to observability, where simple yet important indicators are health checks, and monitoring provides operational indicators. In RecoverIT, the notion can be extended to incorporate logs from applications and infrastructure, metrics, logs from CI/CD pipeline, deployment history, Git commits, pull requests, configuration changes, and information about the service health state. All these can be aggregated in the structure of the incident timeline prior to selecting the remediation playbook by the AI reasoning layer.

The paper emphasizes the significance of automation in minimizing manual intervention and offering recovery consistency. The creation of remediation playbooks could capture the operational expertise and make typical activities consistent and repeatable.The approach is complementary to the procedural one proposed in Flow-of-Action, where structured processes restrict the investigation of LLMs. Also, the approach is complementary to the multi-agent systems in AIOpsLab, LATS-RCA, and GALA where artificial intelligence is used for performing complex diagnostics. Thus, all these studies point to layered architecture, where AI performs reasoning and decision-making and deterministic operational processes control the execution of the actions. The relationship with RCA research is particularly important for RecoverIT.

Health-probe-based self-healing can answer “is the service unhealthy?” and playbooks can answer “what known recovery action should be attempted?”, but reliable autonomous incident handling also requires answering “why did the service become unhealthy?” The papers such as RCAEval, MicroRCA-Agent, GALA, and others try to solve this diagnostic problem through the means of telemetry correlation, multimodal reasoning, graph-based approach, and LLM agents. The approach of RecoverIT uses all these technologies at once by putting evidence-driven RCA first, so that the remediation would be connected with the specific root cause rather than just a generic health failure, triggering it. In addition, the paper speaks about the possibility of implementing different remediation levels, when some actions are to be done automatically while others have higher risks and need more careful consideration.

It is crucial when we consider the Safety Layer of RecoverIT. For instance, the restart of a sick pod can be considered low-risk activity that can be performed without any checks, whereas the roll back of production release and the configuration changes of the infrastructure may be risky and require validation. Such an approach of policy-driven execution helps to avoid an unwise decision from the LLM. For the Autonomous CI/CD Incident Triager & Self-Healer project, the paper is important for the Recovery Planner, Safe Action Executor, and Post-Remediation Verification modules.

## 2.16 CI/CD FAILURE REMEDIATION

In this paper, we present LogSage, an end-to-end Large Language Model based solution tailored to detect, diagnose and fix failures in CI/CD pipelines. The current paper is particularly relevant for RecoverIT since it directly tackles the issue of automated failure analysis in CI/CD pipelines. In contrast to general solutions to the incident management task, LogSage is designed for analyzing specific features of CI/CD pipeline logs, such as their large amount, heterogeneity, noisiness and difficult manual processing. The LogSage framework employs a two-step pipeline. At the first step, the root cause analysis for failed CI/CD pipeline logs is conducted.

Rather than feeding whole raw logs to the LLM, LogSage utilizes a special preprocessing pipeline to highlight the most relevant information contained in log data. This approach reduces the workload for the language model and allows the reasoning module to take into account only the high-quality pieces of evidence. As a result of this step, a diagnosis with an explanation is provided. The second step of the pipeline is remedy generation. LogSage utilizes a Retrieval-Augmented Generation technique to obtain historical remedy plans.

The retrieved data is added to the RCA output so that the system will be able to create a proper remedy to the detected problem. The tool calling method is applied to convert the created remedy to an actual remediation plan. This is crucial for RecoverIT as it shows that the LLM-based system is capable of progressing from just describing the incident to actually creating the actions to fix the issue. One of the significant contributions of the research is that it was evaluated on both a publicly

available dataset and a large-scale industrial one. The RCA step had around 98% precision and almost perfect recall rate, which proves that specialized preprocessing improves the performance of LLMs in analyzing CI/CD logs.

The framework was also deployed in an industrial CI/CD environment where it processed more than 1.07 million executions over a year, demonstrating that LLM-based CI/CD failure management can operate at significant scale. The reported end-to-end precision exceeded 88%, showing the practical potential of combining automated diagnosis with solution generation. For RecoverIT, this paper offers a great basis for implementing a CI/CD failure analysis module, whereby the same technique of pre-processing and filtering of logs can be adopted. Rather than considering the whole CI/CD log as a context, it is possible to identify significant errors, pinpoint specific stages of the pipeline, create a timeline of the incident, and consolidate the findings with other information on deployment and source code. However, a future RecoverIT version could extend the scope of LogSage beyond CI/CD pipeline logs.

LogSage primarily focuses on failures within the CI/CD environment, whereas the proposed system is intended to connect software delivery events with production incidents. For example, the proposed agent can investigate whether a failed deployment, recent Git commit, configuration modification, or dependency update is responsible for a production failure.It is able to correlate CI/CD logs with application logs, infrastructure metrics, deployment history, and service health logs. Another valuable extension that should be considered is hypothesis-based root cause analysis. In this way, instead of taking the first generated diagnosis by the LLM right away, the proposed system is able to generate several potential root causes and check them against available evidence gathered from various sources.

For instance, after identifying the problem of database connection failure, the agent may consider database failure, wrong configuration, network issues, or deployment failure as several distinct hypotheses. Then, additional evidence can be collected to validate which one is more valid. The paper offers a solid base for the proposed Recovery Planner as well. LogSage shows how historical knowledge and tool invocation can be leveraged to create a solution. The proposed system can build on this by implementing a safety layer between the LLM and the infrastructure.

Rollback of deployment, restart of the service, retry of pipeline and scaling can be allowed as well as dangerous actions can require human approval. Another option for the proposed research would be a special safety layer between the LLM and infrastructure, which will add more value. Actions like rollback of the deployment, restart of the service, retry of the pipeline and scaling can be whitelisted, whereas dangerous actions may require human approval. The proposed system can also be expanded through the addition of closed-loop verification. Creation of an action for remedy is not equal to solving the problem at all. Agent should monitor the error rate, health of services, logs, and other metrics after rollback or restart of the application. In case the system starts working properly, it means that the incident is solved.

However, if the failure still exists, the agent should investigate again and redefine its hypotheses. In summary, the main research opportunity offered by this paper is to expand the CI/CD failure diagnosis into a more general evidence-based incident response process. The RecoverIT system uses CI/CD logs, the previous resolution history, git commit messages, deployments, production telemetry, hypothesis-driven reasoning, and verification of hypotheses after remediation. Overall, this is a much broader investigation process than just the CI/CD failure diagnosis process.

The paper introduces LogSage - an end-to-end Large Language Model-based approach intended to detect, diagnose, and resolve failures in Continuous Integration and Continuous Deployment pipelines. It is very relevant to our topic as it solves the problem of automated failure investigation within CI/CD systems. As opposed to a general purpose incident management system, LogSage is targeted at the specifics of CI/CD pipeline logs - these logs are typically long, heterogeneous, noisy, and hard to be analyzed by hand. LogSage suggests a two-stage approach. In the first stage, the system performs root cause analysis on the failed CI/CD pipeline logs.

Contrary to a general-purpose approach when the whole logs are fed into an LLM, LogSage suggests a special preprocessing pipeline to find valuable error information and filter out unnecessary parts of the logs. That alleviates the burden on the language model and allows the reasoning module to focus on high-quality evidence pieces. At this stage, the output of the process is a diagnosis and its rationale. The second stage is dedicated to remedy generation. Here, LogSage leverages the technique of Retrieval-Augmented Generation.

The retrieved information is then merged with the RCA outcome to ensure that the system comes up with an adequate solution to the identified problem. Tool-calling is then used to translate the generated solution into an actionable remediation procedure. This is particularly important for RecoverIT because it demonstrates that an LLM-based system can progress beyond simply explaining an incident and can instead generate actions intended to resolve it. A major contribution of this work is its evaluation in both a public dataset and a large-scale industrial environment. The RCA stage achieved approximately 98% precision with near-perfect recall, demonstrating that specialized preprocessing can substantially improve LLM-based analysis of CI/CD logs.

The framework was also deployed in an industrial CI/CD environment where it processed more than 1.07 million executions over a year, demonstrating that LLM-based CI/CD failure management can operate at significant scale. The reported end-to-end precision exceeded 88%, showing the practical potential of combining automated diagnosis with solution generation. For RecoverIT, this paper offers a great basis for implementing a CI/CD failure analysis module, whereby the same technique of pre-processing and filtering of logs can be adopted. Rather than considering the whole CI/CD log as a context, it is possible to identify significant errors, pinpoint specific stages of the pipeline, create a timeline of the incident, and consolidate the findings with other information on deployment and source code. However, a future RecoverIT version could extend the scope of LogSage beyond CI/CD pipeline logs.

LogSage primarily focuses on failures within the CI/CD environment, whereas the proposed system is intended to connect software delivery events with production incidents. For example, the proposed agent can investigate whether a failed deployment, recent Git commit, configuration modification, or dependency update is responsible for a production failure. It can combine CI/CD logs with application logs, infrastructure metrics, deployment logs, and health information of services. An important extension of the framework is hypothesis-based root cause analysis. This means that the framework will not only rely on the first cause given by the LLM but will rather generate a number of different possible causes that can be tested based on the evidence collected from different sources.

For example, after detecting a failure in a database connection, the framework may analyze database outage, misconfiguration, network issue, or deployment as different

hypotheses. After that, it can collect additional evidence in order to understand which root cause is the most likely one. This paper can also serve as a good basis for developing the proposed Recovery Planner. LogSage shows how historical knowledge retrieval and tool calls can be leveraged to perform effective remediation. The proposed framework can use this approach and create a safety layer between the LLM and the infrastructure.

Some examples of actions to be whitelisted can include deployment rollback, service restart, pipeline retry, and controlled scaling. At the same time, potentially destructive actions will have to be approved by humans. This method can also be enhanced by the addition of a special safety layer between the LLM and the infrastructure. Actions like deployment rollback, service restart, pipeline retry, and scaling can be whitelisted, while potentially destructive actions will have to receive human approval. In addition, the proposed system will be able to enhance the research by including closed-loop verification. Making an action to fix a problem is not an automatic solution of that issue. The agent needs to verify error rates, service health, logs, and other data after rollback or restart of the application. If the system becomes operational again, then the incident is solved.

If there is still some failure occurring, then the agent needs to continue investigation and improve the hypotheses.

In summary, the main research opportunity offered by this paper is to expand the CI/CD failure diagnosis into a more general evidence-based incident response process. RecoverIT leverages CI/CD logs and the history of resolutions along with Git commits, deployment events, production telemetry, hypothesis-driven reasoning, remediation and testing of hypotheses, and post-remediation validation. In essence, it is a more general investigation and recovery process compared to the CI/CD failure diagnosis one.

## 2.17 CLOUD OBSERVABILITY RCA

The current paper proposes a solution that combines Large Language Models with cloud-native observability platforms to enable automatic root cause analysis and remediation in the cloud environment. This solution is highly relevant to the RecoverIT work because it covers the whole pipeline of the process starting with telemetry collecting and anomaly detecting up to root cause finding and automatic

recovering. There is one main problem identified in the modern cloud-native solutions: applications are becoming more complex and consist of many microservices, containers, and other distributed components. Hence, there is plenty of operational data produced in the form of metrics, logs, and traces. Rule-based monitoring solutions can identify certain known situations, but they lack the capability to comprehend the complex relationships between signals.

Only those anomalies and evidences that can potentially be related to the issue will be sent to the reasoning layer. The main element of the framework is the enhanced root cause analysis layer using the LLM. Domain-specific LLM is used to perform the reasoning on the data and propose root cause hypotheses. The system is designed to give the explanation for the diagnosed issue rather than just the fault name. This requirement matches the RecoverIT one stating that each root cause hypothesis should be backed by the evidence (logs, metrics, deployments, source code changes).

The paper also proposes a domain adaptation method for LLM.

Operational data may include specific jargon, system behavior, and failures that are underrepresented in language model training data. That is why the proposed framework utilizes the operational data in order to fine-tune the model and to increase its reasoning capabilities concerning cloud-native incidents. Thus, the importance of domain-specific knowledge in application of LLMs to AIOps and incident management becomes apparent. Yet another important element of the framework is the automated remediation layer.

After the root cause of the problem has been recognized by the LLM, a remediation plan is generated and translated into the operational actions. It is important to note that the architecture includes safety measures in regards to remediation of problems, not arbitrary executing of actions generated by the model. Such an approach is highly relevant to the RecoverIT project, because infrastructure actions have to be limited in some way or another. At the same time, the framework has an important feedback loop between the remediation and system monitoring layers.

This is an important characteristic of a self-healing system because simply executing a recovery command does not guarantee that the original incident has been resolved. The system must verify whether the relevant symptoms have disappeared. Experimental evaluation reported substantial improvements in incident-management

performance. The proposed framework achieved approximately 95% F1-score for anomaly detection, an 84.2% reduction in mean time to resolution compared with rule-based approaches, and an automation rate of approximately 91% for common incidents. These results demonstrate the potential of combining traditional observability techniques with LLM-based reasoning and automated remediation.

For RecoverIT, this paper provides the foundation for the production observability side of the architecture. Just like the proposed system can do, it is possible for the system to gather metrics, logs, and traces to be used as evidence during incident investigations. Nevertheless, the difference between the proposed system and a typical cloud-native self-healing system is that, unlike the latter, the former does not just gather runtime operational signals. It also takes into consideration the signals related to the software delivery process including Git commits, CI/CD execution results, deployment versions, configuration changes, and pull requests.

For instance, in the case where errors start emerging right after a deployment, the agent should do more than check the runtime telemetry. It should analyze the changes introduced by the new deployment, compare it with the old healthy deployment, review CI/CD artifacts, and decide if it is possible to attribute the problem to the deployment. Multimodal evidence used by RecoverIT is further supported by the research paper as well. Logs, metrics, and traces can be viewed as complementary representations of the same incident. a future version of RecoverIT can take advantage of multimodality by adding yet another kind of evidence – the software-delivery context.

Namely, git history, deployment artifacts, CI/CD results, configuration and release artifacts can be correlated with runtime telemetry and create a full incident timeline. Another possibility would be the use of risk-aware remediation. In this case, the recommended system will have to classify the recovery attempt depending on the risk it poses to the system. Low-risk recoveries such as restarting the service or retrying the pipeline will have to be done automatically, while high-risk recovery attempts such as database and infrastructure changes will have to be manually confirmed.

## 2.18 COMPARISON AND RESEARCH GAP

| Research strand | Contribution | Remaining gap for this project |
|---|---|---|
| LLM agents | Iterative tool use and evidence retrieval | Need bounded cost and structured provenance |
| Log-focused RCA | Filters high-value log evidence | Need to join software delivery with runtime context |
| Microservice RCA | Causal and multimodal reasoning | Need source identity, timeline and abstention |
| CI/CD failure analysis | Pipeline-specific diagnosis | Need production incident correlation |
| Self-healing systems | Recovery procedures and monitoring | Need approval and execution safety before deployment |

Table 2.1 Literature comparison

The gap here is that of combining delivery event and runtime observation data in one evidence trail while ensuring the accountability of the conclusions and recommendations made. RecoverIT has managed to bridge this gap in the areas of investigation and recommendation. The system does not purport to beat other RCA efforts on either accuracy or operational efficiency since there is no labeled incident benchmark.

CHAPTER 3

# METHODOLOGY AND SYSTEM DESIGN

## 3.1 DESIGN PRINCIPLES

Inference shall be done only after observations have been made. Each observation is backed with a source, identifier, and timestamp. This makes sure that a hypothesis can use it as a concrete record. The model does not validate whether a record referenced by the hypothesis is actually existing. We notice that this principle is used when source types, normalizing sources, creating contexts. Finally, all policies are reviewed.

The configuration during the runtime is clear. The settings file provides the selected reasoning provider, the model. It also provides the available source adapters, the timeouts, and the checkpoint directory. An automatic live composition needs a concrete provider to work with it. The fake providers are injected only by the tests themselves, but not the system. Such a configuration will prevent unintentional running of production code with the test data.

The investigations have defined limits within the system. The graph uses the investigation budget and stopping criteria for managing the rounds of collection. The investigation is not automatically completed because of the acceptable claim made by the LLM. If there is not enough information to make a diagnosis, the system can produce an inconclusive result where criteria are still unfinished.

## 3.2 SYSTEM ARCHITECTURE

There are three main levels of the application. The boundary layer accepts the canonical incident and identifies all of the configured source capabilities. The evidence level collects, validates, normalizes, archives, and correlates the evidence in a timeline. Finally, a context capture. The reasoning level identifies gaps, makes queries and hypotheses.

Then it examines stopping conditions, ranks the hypotheses by evidence, and optionally creates a remediation recommendation. The runner converts the graph state to an immutable state appropriate for CLI and HTTP clients.

The LangGraph state is designed to contain only information that is easy to serialize and deserialize, i.e. no services. Dependencies are separated from the state of the graph and are passed to the nodes as needed. It means that you can checkpoint your work without converting connections to HTTP, callbacks, or any access utilities to something serializable.

In case of practical implementation, there is a SQLite checkpointer handling saving of the state and for tests, there is a state using memory.The adapters of the sources are read-only. It gives an ability to perform the investigation with the necessary permissions.

The agent does not change anything in the sources itself. The adapter exposes what is possible to query and returns a result with the source's status, records and warnings. The sources which are unavailable are stated as gaps and returned without fabricating any evidence.




Figure 3.1 RecoverIT investigation workflow

## 3.3 INCIDENT INTAKE AND CONTRACTS

IncidentSeed contract contains all data fields such as incident id, service impacted, environment, severity, time detected and received summary. Pydantic typed contracts are used to ensure the validity of data being transferred between different modules. Versioned schemas and canonical json minimize ambiguities between collection, evidence, timeline, and reasoning modules. The source capabilities catalog indicates what data can be provided by the adapter. A query plan asks for certain fields and time ranges.

Then, the collection service executes these queries taking into account the number of parallel queries and their duration. It finally reports on the success and failures of queries. In this way, you are able to distinguish the absence of the data due to adapter failure and success of the query.

In this case, the role of the contracts becomes essential since it is an AI-assisted system. The output of the free-form model is created according to the requirement. Identifiers, time fields, evidence references, and status enums validation in advance makes invalid data an error or non-blocking recommendation.

## 3.4 SOURCE COLLECTION

Adapters for configured sources provide read-only access to specified source types. These source types consist of logs, metrics, Git changes, deployments, pipelines, and health indicators. Live runtime generates adapters for only those sources which are under their trusted configuration settings.

Instead of assuming that each incident could use all sources, capabilities provide the ability to adapt the query planner to a particular deployment.

The collection process is split into two processes including retrieving and interpreting. Adapters retrieve the original record IDs and timestamps but do not find the root cause of the problem. The collection service merges queries and also handles partial failures.

Besides, it provides the assurance that the analysis will not fail if any of the sources times out. It will assist in the latter phases to assess the collected evidence and whether there is enough of it or not.

The source endpoints and credentials are included only in server-side configurations. The operational data is viewed as untrusted data and not as a command for the graph. Record should be validated and redacted prior to reaching the model as it is vital since logs and configuration payloads could contain the data from adversaries.

## 3.5 EVIDENCE PROCESSING AND PROVENANCE

Normalization of the timestamp into UTC format while retaining the event time and observation time is performed at the evidence layer. An ID for the record, classification, and record quality are provided by this layer. Source IDs are stored for traceability reasons. These operations enable comparison of information from different sources and tracing of data origins. Deduplication deals with duplicate events. For example, duplicate lines in a log file should not result in separate votes on the same hypothesis. Evidence Repository helps to store and filter records as well as provide sorting and pagination of the results in a consistent manner. This process

allows replicating the investigation. Redaction occurs before the summation of data for reasoning. No secrets and tokens should appear in the prompts and recommendations. The incident context includes evidence summaries and source coverage. This helps the hypotheses layer to highlight supported assertions and missing pieces of information.

## 3.6 TIMELINE AND CONTEXT CONSTRUCTION

All normalized evidences are linked to their associated event and sorted according to their event times by the Timeline builder with a tie-breaker mechanism. The time links are based on the order of occurrence of events and relations between them, but it does not indicate any causation relationship between them.

This is significant since although the deployment before error can be considered as an important piece of evidence, it does not mean that there is any causation relationship due to this order.

Context constructor combines the incident seed, source coverage, evidence summaries, timeline events and relationships into a snapshot. This snapshot is what is given to the reasoning services. This prevents those services from reaching back into raw source adapters arbitrarily after context publication. Contextual information is needed to ensure that the reasoning service has enough information to make decisions but at the same time the amount of contextual information should not be too limited. This is because insufficient contextual information will lead to a hypothetical diagnosis, while too much information leads to lack of causal evidence. That is why the graph begins by deciding what information is needed then planning queries. It can update the context after a collection round and decide whether another round would be informative.

## 3.7 INVESTIGATION GRAPH AND STATE

The built-up graph consists of initialization nodes, gap assessment, query planning and evidence gathering. It also consists of context building, hypothesis generation, stopping evaluation, ranking and incomplete conclusion nodes. If the remediation planning service is available, completed and incomplete flows will reach final plan node. Flow that is marked as incomplete will go to escalation and verification guidance but not make any production change recommendation.

Conditional edges guide the flow according to the results of query planning and stopping evaluations. Graph may continue with evidence gathering in case of existence of remaining budget or conclude with the received best result.

Checkpoints that were created and stored have the incident id as a key. Runner is able to create a new graph run or resume the existing one. Progress events are delivered through callback that belongs to a specific task, but not to the state of the graph itself. It avoids situations when different investigation runs could accidentally deliver incidents' events to other clients. Runner interprets graph results as a deterministic result which contains status, hypotheses, timeline, evidence, budget consumption and remediation guidance.

## 3.8 HYPOTHESIS RANKING AND STOPPING

Hypothesis candidate has an assumed cause, affected component, evidence, contradictions, and questions to be answered. The ranking engine derives a reproducible score by summing all evidence supporting the candidate and subtracting contradictory evidences.

Source independence guarantees that the same evidence is used only once in calculation. Symptom coverage determines whether the assumed cause explains the symptoms; temporal consistency addresses the sequence of events and change consistency deals with the direct delivery evidence.

Contradiction and lack of causation decrease the confidence level. Strong correlation between the symptoms does not necessarily indicate knowledge about the cause. For that reason, both symptom confidence and causation confidence are reported independently. Ranking is only possible if there is the possibility to check the evidence; thus, the score is included in the final report. The stopping criteria include a remaining budget, sufficiency, and quality of collected data. Future reasoning may refuse to provide any results at all. This step tries to avoid the presentation layer converting the weakly ranked hypothesis into the root cause since it should have something on top of the list.

## 3.9 RECOMMENDATION-ONLY REMEDIATION

The remediation service obtains a candidate plan in typed form from the configured reasoning provider. The plan consists of requirements, descriptive actions, expected

results, verification, reversal actions, and risk classification. This service offers advice regarding the problem, and the output requires human approval. None of the commands are executed in this situation.

Deterministic policy does not allow diagnostics or evidence IDs that cannot be verified or are mutually inconsistent, executable shell commands, and credentials. In case the provider does not provide any result within the set timeouts, it means that there is a blocked failure. In case of uncertain reasoning, there is a blocked plan with escalation advice.

The system is specifically narrowed when it comes to the execution of suggested remediations because it enables a safe interaction of services without making any assumptions about the availability of the approval, execution, and post-execution check mechanisms.

## 3.10 INTERFACES AND DEPLOYMENT

The CLI takes the runtime configuration and incident file as inputs and creates an investigation report. The FastAPI server allows the same internal runner to be accessed via HTTP requests. The web UI displays status, progress, evidence, and results ranked. All of these are presentation layers on top of one application output, not separate diagnostic engines.

Live configuration defines providers and source adapters; the configuration that does not pass validation is rejected. Checkpoint storage can use an application data directory path; each execution is uniquely identified based on incident identifier. It allows the framework to be used in live tests and integrations, assuming access to the external resources.

Development environment requirements are Python 3.11 or newer and typed dependencies such as Pydantic, FastAPI, httpx, LangGraph and SQLite checkpointing support. Requirements for testing environment are different.

## 3.11 EVALUATION DESIGN

At the moment, the repository provides the fact that everything is fine by means of unit testing, contract testing, and integration testing. In order to carry out a complete test of the root cause analysis, you will need incidents characterized by true root causes, understanding about the origin of the sources used and consistency of the

testing conditions. Metrics will assess the accuracy of the root cause identification, the accuracy of the source citation, the effectiveness of the system in terms of its ability to identify the cases when the answer should not be provided, the time needed to investigate the problem and its cost.

In order to carry out an unbiased test, the baseline based on rules or manual one should be tested on the same incidents and snapshots of the sources used. This test should be performed separately on incidents with missing sources, contradictions in the logs and excessive log information. Remediation recommendations will be assessed depending on their evidential base, safety, clarity and verification. All these points should be considered prior to executing the recommendation.

| **Module** | **Input** | **Output** |
|---|---|---|
| Collection | Incident, source capabilities, query plan | Raw evidence batch |
| Evidence processing | Raw records | Normalized evidence and repository entries |
| Context | Evidence and incident | Timeline and context snapshot |
| Reasoning | Context and budget | Candidate hypotheses |
| Ranking | Candidates and evidence | Ranked hypotheses or inconclusive status |
| Remediation | Ranked output and context | Advisory plan or blocked guidance |

Table 3.1 Module interfaces

| **Boundary** | **Control** |
|---|---|
| External operational sources | Read-only registered adapters |
| Untrusted record content | Validation, normalization and redaction |
| LLM output | Typed contracts and deterministic ranking/policy |
| Production changes | No write-capable executor in current runtime |

Table 3.2 Safety boundaries

CHAPTER 4

# IMPLEMENTATION AND RESULTS

## 4.1 REPOSITORY IMPLEMENTATION

The architecture includes contracts, collection, evidence, timeline, investigation graph, reasoning, remediation, and the RecoverIT software. Metadata on the package consists of Python 3.11 or higher and Pydantic, httpx, FastAPI, Uvicorn, LangGraph, and SQLite checkpoints. The README contains instructions for installation and the CLI usage. The source code files consist of live paths code, while the docs directory contains design choices, interfaces, and handoffs. There is also some legacy architecture documentation on the larger system that was supposed to be created; this chapter will describe the parts that have been implemented.

## 4.2 RUNTIME COMPOSITION AND GRAPH

Module `recoverit.composition` performs validation of the runtime JSON file and constructs the graph using the selected source adapters and actual reasoning provider. Automatic construction of the test and benchmarking modes is forbidden. Live mode uses SQLite-based checkpoint saver except when another one is passed. Module `investigation.graph.builder` constructs ten named nodes and directs all completed/inconclusive investigations towards the planning recommendation when configured.

The `recoverit.runner` module invokes the graph with a canonical incident and investigation budget. It exposes status, stop reason, ranked hypotheses, timeline events, evidence, provider identity, budget usage and optional remediation plan. Its Markdown report shows a factor-level score breakdown and labels remediation as a human-reviewed suggestion. A resume method loads checkpointed graph state for an incident ID.

## 4.3 COLLECTION AND EVIDENCE

Collectors provide read-only adapters for specific sources and also the collecting service itself. Evidence is validated and normalized, and the time stamps are recorded along with the identity information, evidence is re-inspected, duplicates are removed,

and all of the above information is stored via repository interfaces. Timeline services relate pieces of evidence with events and relations.

The difference is observed in package and test boundaries. In doing so, it prevents the payload of the adapter from being used as a conclusion. Data on failures are still available depending on the status of the source and the warning messages; therefore, the absence of the source will not give a false negative.

## 4.4 REASONING AND RANKING

The reasoning module includes the following elements: provider interface, live client, hypothesis service, ranking module and termination conditions. The test mode providers and dummy recorded providers can produce repeatable graph behavior, but live composition will not select such providers implicitly. The ranking module is responsible for computing the following elements: support by independent sources, symptoms, consistency in time, change in evidence, specificity and prediction support.

This will eventually provide the score, as well as the factors that have been used to generate the score. This is necessary because the reader is able to evaluate why a particular cause ranks higher than another. However, the ranking will still be influenced by the model and level of evidence used.

## 4.5 REMEDIATION POLICY

The current remediation module allows for the creation of recommendation-based plans only. This module requests a candidate plan from the reasoning provider and validates the plan against deterministic policy. The validation module rejects plans that are of low confidence, contradictory, unknown evidence ID, and use executable commands.

This will be a designed plan that comprises of the steps needed for a human to execute and even undo the solution where necessary. The design will not comprise of anything like issuing tickets, tools and/or command execution.

## 4.6 INTERFACES

CLI will carry out a designed investigation using the JSON data file for the particular incident and generate a Markdown document. HTTP service and web client will exhibit the same result for the application. The interface has to be seen as a reflection

of the investigation process whereas the evidence log and type graph are the actual artifacts.

Addition of ranking of the hypotheses, scoring of the evidence, factor analysis of the evidence, unresolved criteria where necessary, and suggestions on how to solve problems are part of the report compiled by the runner. This is done to enable the reviewer to make a comparison between the two.

## 4.7 TESTING AND VALIDATION

In order to confirm that the investigation process of a live case is correct, a dedicated case study was performed. It involved an installation called HarborShop that runs the checkout application in a local mode. In one of the revisions, the directory with SQLite was pre-provisioned with the data for products and orders. In another revision, a one-liner was introduced that pointed to the other directory that was not created yet. Thus, the bug was known and could be compared with the identified reason of the bug from RecoverIT.

The same traffic script was executed in both revisions. For a healthy revision, 15 checkout requests returned HTTP 201 status code while readiness request returned HTTP 200. But after the modification was done, all 15 checkout requests returned HTTP 500, readiness returned HTTP 503. The liveness requests still returned HTTP 200, but the invalid requests returned HTTP 400, as expected. The logs of the application and the database include the information about failed requests and inability to open the database file.

Following this, RecoverIT was performed using the settings below: local Git changes, version-controlled configuration file, log files, HTTP check and request measurement as evidence source types. The entire investigation process took 16.45 seconds. The investigation trace and incident summary from the tool interface is provided below.




Figure 4.1 Live investigation trace & incident summary

The path change for the order storage issue was classified first as a configuration regression problem with evidence score 76.45/100 points. Infrastructure failure was classified second with 49.70 points. The top-ranked cause was consistent with both one-line code modification and readiness checks failure. The ranking and evidence results are shown in Figure 4.2 below. These percentage scores are the internal ones and do not indicate the probability of the correct diagnosis. Particularly, the symptom confidence score is low because of individual log scoring.

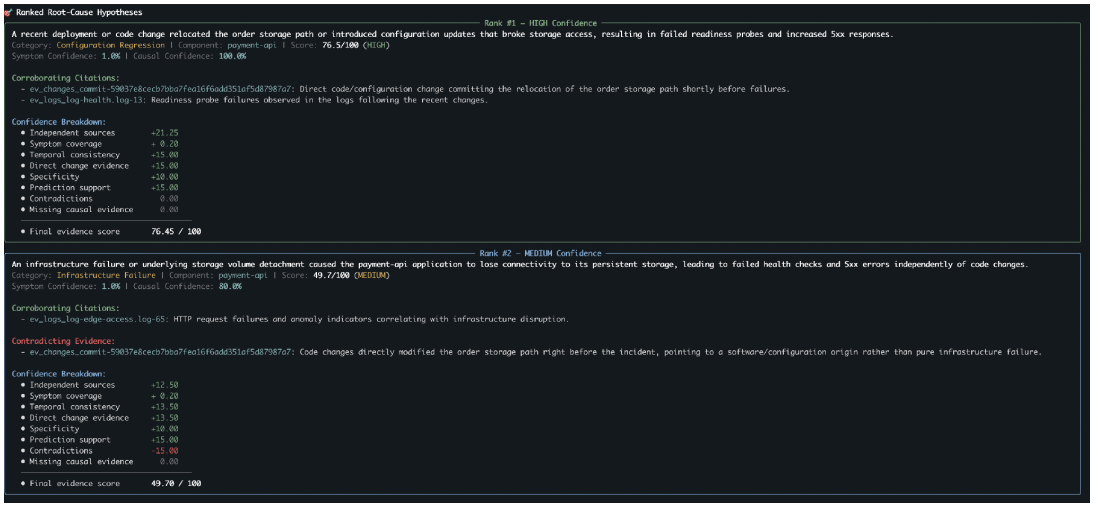



Figure 4.2 Ranked root-cause hypotheses for the controlled incident

It also produced an ordered sequence involving the review of the configuration, readiness check and logging, fix for the path, and recovery validation. These are illustrated in Figures 4.3 and 4.4. One must remember that all of these are just suggestions and require human examination. Nothing was automated by RecoverIT and post-recovery performance was not tested. Some references are made to central dashboards and pipelines which are mere suggestions since the use case involves local logging and service.

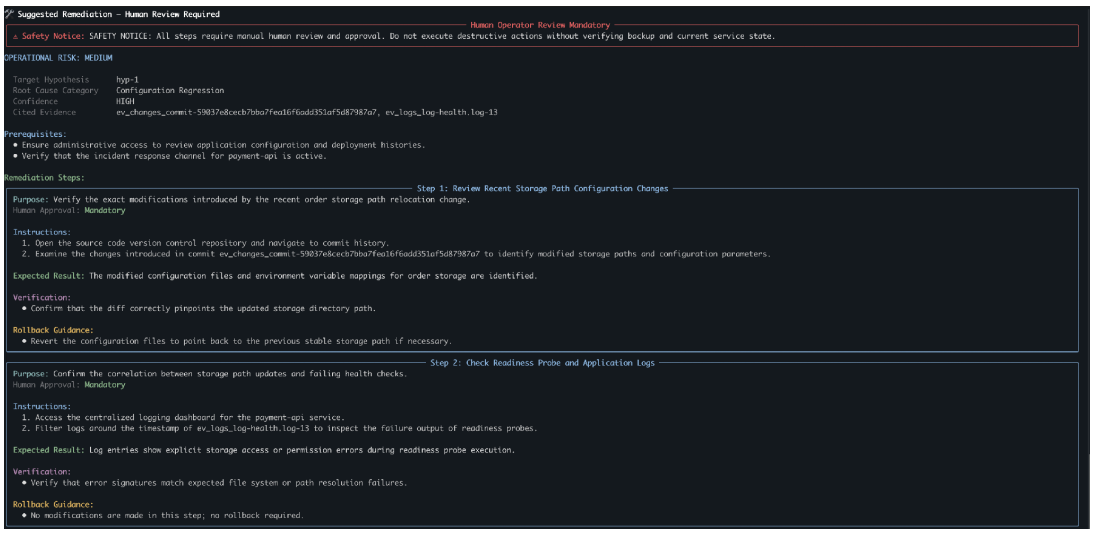



Figure 4.3 Advisory remediation output with review & log inspection steps

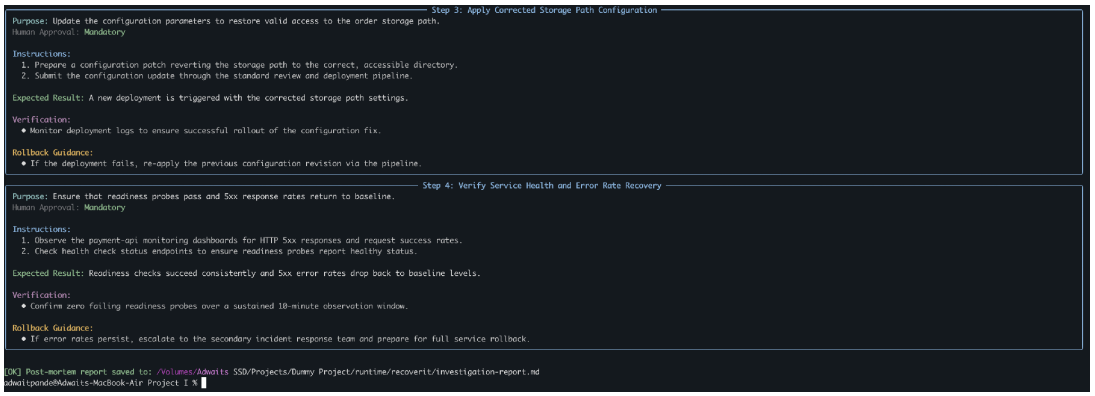



Figure 4.4 Advisory configuration & recovery verification steps

The above scenario is evidence that the workflow employed was capable of identifying the introduced failure in a managed environment using live data from different sources. It is, however, not evidence of accuracy in an unknown environment or the time taken to solve the problem. The above scenario is, therefore, evidence of an extended case study of the test result in the repository test.

## 4.8 RESULTS ANALYSIS AND LIMITATIONS

This leads to a cohesive and validated investigation pipeline with clear types of boundaries and safe recommendations for advisory remediation. Tests verify that the developed components act as intended according to their assumptions. Nevertheless, they do not prove how accurate is the analysis that RecoverIT performs when dealing with new production issues. In this case study, there were no comparisons made with other software or even a human in order to evaluate its performance or the reduction in the mean time to resolution.

There are certain limitations for this solution since it lacks source code coverage, accuracy of the telemetry data, the way how providers act and plausibility of the alternative hypothesis. Ranking using deterministic algorithms assists in overcoming some potential failures, but does not replace the labeled training process. Moreover, the policy component allows avoiding some unsafe recommendations; however, the security analysis and approval of the recommendations are necessary prior to the execution.

Initially suggested name regarding the self-healing is only an example of the future direction. The present solution finishes with the generation of recommendations, which should be then reviewed by a human. It includes no execution component and therefore, the verification of recovery is not provided. The limitation is stated explicitly so that the readers could see the distinction between implemented solution and the literature examples.

CHAPTER 5

# CONCLUSION AND FUTURE WORK

## 5.1 CONCLUSION

The RecoverIT process is presented as bounded in terms of incident investigation in CI/CD environment. It receives the read-only source records which are configured as an input; it builds evidence from the sources, timeline and the incident context, creates root cause hypotheses and orders them according to their priority based on evidential reasons behind each one.

The process represented by LangGraph has explicit transitions and checkpointing. The output is designed for review by the operator and cannot be run as such in an automatic way by the model.

Current implementation version of the system provides guidance to the remediation based on the deterministic safety rules. The change recommendation is not possible when there is a failure either at confidence or content safety check stage. This is the key safety implication of the work. The assistant is able to produce recommendations structurally while maintaining human in the loop.

There is still much to do. The ability to determine root causes properly, solve incidents quicker, operate safely and self-heal are left untouched.

## 5.2 FUTURE WORK

First of all, there must be a labelling of the incident evaluations. Incidents should have the fixed root cause, multiple sources, partial information, and sometimes even have contradictions between the observations. The incident evaluation should be able to check for the top-one and top-k accuracy, accuracy of the evidence, accuracy of the reference, calibration, abstention and provider costs. In addition to the machine learning part, there must be a manual labelling in order to make a fair comparison.

There are several possible approaches regarding further research concerning the inclusion of additional sources and improvements of causal inference. The inclusion of additional sources, dependencies, and the changes in deployment would make the context of the system broader. It is important to conduct tests that will show that the use of additional sources makes the diagnosis of incidents better than the diagnosis

based on additional training data. It is important to test prompt ingestion, data leakage, and complex event ordering as well.

For the further development of recovery functionality, there are several important factors that should be taken into account. Firstly, there should be a list of operations that were approved beforehand. Secondly, there should be a procedure for the identification of the user who performs changes and the confirmation of his/her authority to perform them. Thirdly, the request for the confirmation of changes should be made before the user performs the changes. Fourthly, there should be the limitation of the distance and time of changes. Fifthly, there should be a possibility to undo performed changes. And finally, the confirmation of the successful performance of the change should be performed. Then only the self-healing becomes possible.